

Multi-User Linux Server Environments

OS Resource Management, File & Device System Calls,
and Kernel-Userspace Interaction

Presented by
B. Ganga Jayadev Reddy

Introduction

01

OS Resource Management

How the kernel arbitrates CPU, memory, files, and devices among many simultaneous users.

02

File & Device System Calls

The exact API surface — open, read, write, ioctl — that processes use to reach hardware and storage.

03

User ↔ Kernel Interaction

Mode switching, traps, and context switches that keep untrusted processes safely isolated from the kernel.

One Kernel, Many Simultaneous Users

A shared Linux server lets multiple people log in at once — over SSH, samba, or NFS — each running independent processes, editing files, and touching shared devices. The kernel is the single arbiter that must keep every user's activity isolated, fair, and secure.

Isolation:

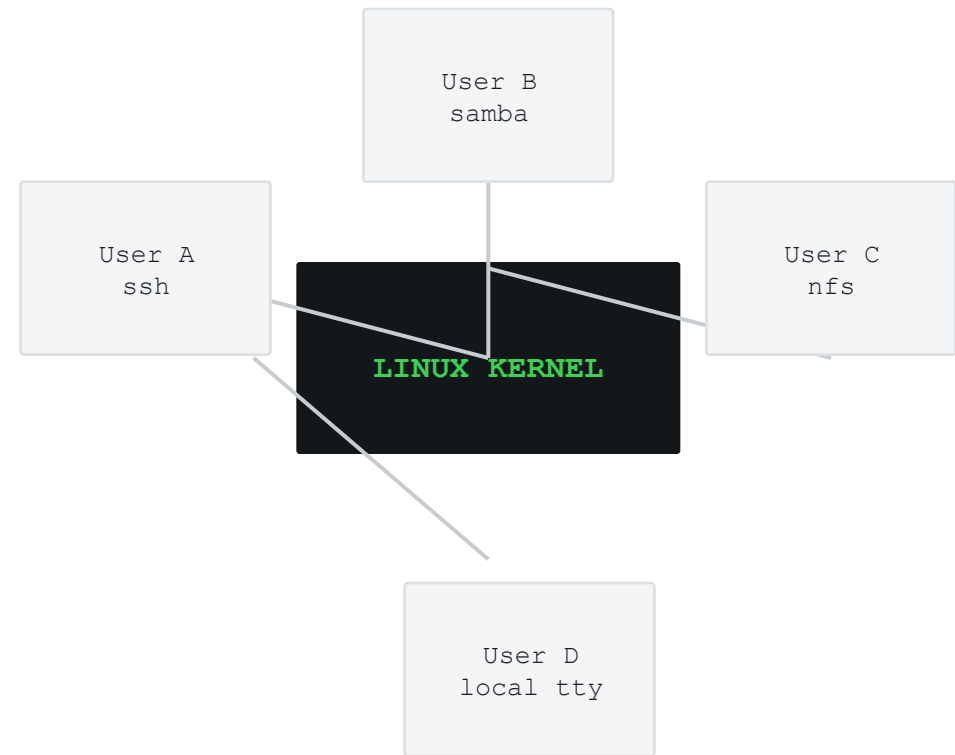
Separate process address spaces &
UID/GID-based file permissions

Fair Scheduling:

CPU time-sliced across every user's
processes

Controlled Sharing:

Common files & devices exposed through
permission checks



OS Functions in Resource Management



Process Management

Creates, schedules, and terminates processes; time-slices the CPU fairly across every logged-in user.



Memory Management

Allocates virtual address spaces, pages memory in/out, and prevents one process from reading another's memory.



File System Management

Maintains directories, inodes, and permissions so files can be shared safely between accounts.



Device Management

Mediates access to disks, terminals, printers, and network interfaces through drivers and device files.

Process Management & CPU Scheduling

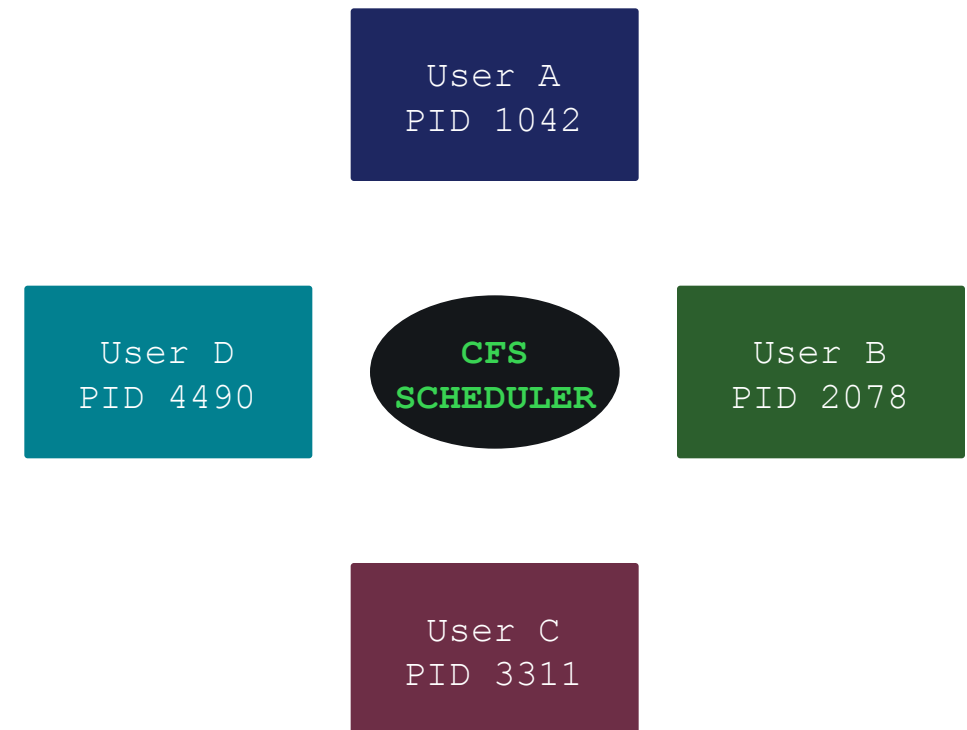
Every login shell, editor, and daemon is a process the kernel tracks in a process table (PID, owner UID, state, open file descriptors). The Completely Fair Scheduler (CFS) then time-slices the CPU so no single user's workload can starve another's.

```
fork() / exec() spawn and replace  
process images
```

```
Each process gets an isolated  
virtual address space
```

```
nice / priority values weight CPU  
share per user
```

```
wait() / exit() manage process  
lifecycle & reaping
```



Memory Management & Isolation

The kernel gives every process a private virtual address space, translated to physical RAM through page tables. This means User A can never read or corrupt User B's memory — even though both run on the same box.



Virtual Addressing

Each process sees a full private address space; the MMU + page tables map it to physical frames.

Protection Bits

Read/write/execute permissions per page block cross-process tampering and code injection.

Paging & Swap

Inactive pages move to swap space, letting many users' processes coexist beyond physical RAM.

File System Management for Shared Storage

Files are the primary sharing surface in a multi-user server. The kernel tracks ownership and permissions on every inode, so directories, documents, and sockets can be shared deliberately — never by accident.

- Owner / group / other permission triad (rwx)
- UID & GID checked on every file access
- Access Control Lists (ACLs) for finer-grained sharing
- Shared mounts (NFS/Samba) re-apply the same checks

```
$ ls -l /srv/shared/

-rwxr-x---      alice      devteam      deploy.sh

-rw-rw-r--      alice      devteam      notes.md

-rw-r-----      bob       billing      invoice.c
sv

drwxrwxr-x      root       staff        public/
```

owner | group | others get no listed permission unless granted

Device Management

Linux exposes hardware as files under `/dev`. Drivers register with the kernel and handle read/write/ioctl requests, so applications use the same file-style calls whether talking to a disk, a terminal, or a printer.

`/dev/sda1`

BLOCK DEVICE

Disk partition — buffered,
random-access I/O

`/dev/tty1`

CHARACTER DEVICE

Terminal session — stream-based I/O

`/dev/usb/lp0`

CHARACTER DEVICE

USB printer — sequential output
stream

`/dev/null`

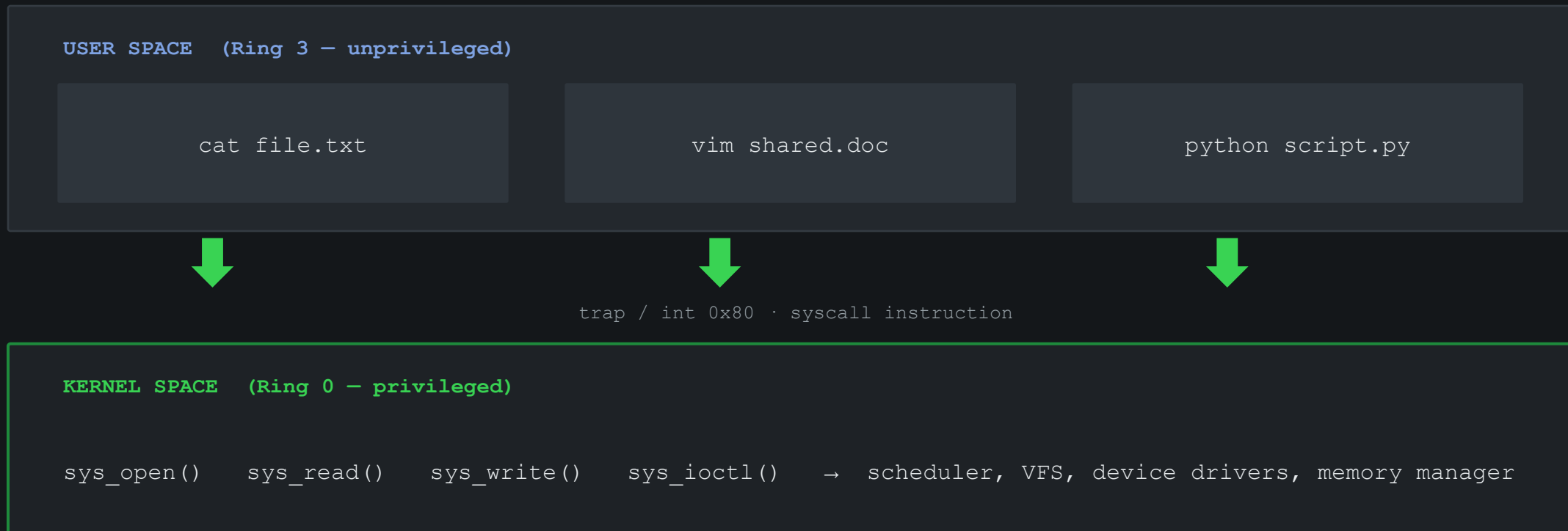
CHARACTER DEVICE

Discards all writes, returns EOF on read

Because devices appear as ordinary files, the same permission model (owner / group / other) governs who may read a tape drive, write to a printer, or access a serial port — unifying storage and hardware security.

The System Call Interface

A system call is the only doorway between an unprivileged user process and the kernel. Processes can't touch hardware or another user's data directly — they must ask the kernel through a well-defined API.



Core File System Calls

Call	Purpose	Typical Multi-User Use
<code>open()</code>	Get a file descriptor; kernel checks permission bits first	Opening a shared config file for read-only access
<code>read()</code> / <code>write()</code>	Transfer bytes between kernel buffers and process memory	Two users appending to a shared log concurrently
<code>close()</code>	Release the file descriptor and flush buffers	Ending a session cleanly after an edit
<code>stat()</code> / <code>fstat()</code>	Query metadata: owner, size, permissions, timestamps	Checking who owns a file before overwriting it
<code>chmod()</code> / <code>chown()</code>	Change permission bits or ownership	Admin granting a team read/write access to <code>/srv/shared</code>
<code>flock()</code> / <code>fcntl()</code>	Lock a file to coordinate concurrent access	Preventing two processes writing the same record at once

Core Device System Calls

Call	Purpose	Typical Multi-User Use
<code>open("/dev/...")</code>	Open a device node like any other file	Opening a serial port or a shared scanner device
<code>ioctl()</code>	Send device-specific control commands outside read/write	Setting a terminal's baud rate or querying disk geometry
<code>mmap()</code>	Map device or file memory directly into process address space	Fast shared-memory access to a video framebuffer
<code>poll()</code> / <code>select()</code>	Wait efficiently on multiple file/device descriptors	A server handling many users' sockets at once
<code>read()</code> / <code>write()</code>	Stream data to/from a character or block device	Sending a job to a shared network printer

How a System Call Actually Crosses the Boundary

1

User process calls a libc wrapper

e.g. `read(fd, buf, n)` in application code

2

CPU traps into kernel mode

syscall instruction switches privilege ring; args placed in registers

3

Kernel dispatches via syscall table

Looks up the syscall number, jumps to `sys_read()`

4

Kernel performs the privileged work

Checks permissions, talks to VFS / driver, copies data to kernel buffer

5

Result copied back, CPU returns to user mode

Process resumes at the instruction after the trap, with return value

The Kernel Is the Trusted Middleman

Resource management scales isolation

Process, memory, file, and device subsystems each enforce per-user boundaries on shared hardware.

System calls are the only sanctioned path

open/read/write/ioctl let applications reach files and devices without touching hardware directly.

Mode switches keep the system safe

Every request traps into kernel mode, gets validated, then returns control — protecting all other users.

Thank You...